SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: **CSA1404** COURSE NAME: **Compiler Design**

COURSE OUTCOME COVERED

C02: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

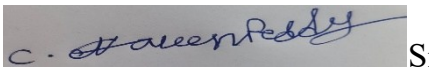
Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:50

Annexure A
DESIGN TASK

Code of Conduct:

I **CHINTHAPARTHI NAVEEN REDDY/192525206 (Name / Reg No)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. / understand that any violation of academic integrity rules will result in disciplinary action.


Signature of the student:

BASIC BLOCKS, FLOW GRAPHS & CODE OPTIMIZATION TECHNIQUES

DESIGN TASK

Submitted in the partial fulfilment for the Course of

CSA1435-COMPILER DESIGN

to the award of the degree of

BACHELOR OF ENGINEERING
AL&ML
Submitted by

C.NAVEEN REDDY
(192525206)

Under the Supervision of

Dr G ANITHA

S 1 M ATS
ENGI N EERI NG

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JULY- 2026

Basic Block Identification

Introduction

A basic block is a maximal sequence of consecutive three-address statements in which control enters only at the first statement (no jumps into the middle) and leaves only after the last statement (no jumps out except possibly at the end). Basic blocks are identified using the concept of leaders.

```
1.              a              =              b              +              c
2.           if           a           <           d           goto           5
3.              e              =              a              +              d
4.                                    goto                                  6
5.              e              =              a              -              d
6. print e
```

(i) Leader Identification

A statement is a leader if it satisfies any of the following rules:

- Rule 1: The first statement is always a leader -> statement 1.
- Rule 2: Any statement that is the target of a conditional or unconditional goto is a leader -> statement 5 (target of 'goto 5') and statement 6 (target of 'goto 6').
- Rule 3: Any statement immediately following a goto / conditional goto is a leader -> statement 3 (follows 'if a<d goto 5').

Leaders identified: statements 1, 3, 5, 6.

(ii) Basic Block Partitioning

Each basic block starts at a leader and extends up to (but not including) the next leader:

- B1: statements 1–2
- B2: statements 3–4
- B3: statement 5
- B4: statement 6

(iii) Block Numbering

```
B1:           1.           a           =           b           +           c
    2.          if          a          <          d          goto          5

B2:           3.           e           =           a           +           d
    4.                                 goto                                 6

B3:           5.           e           =           a           -           d

B4: 6. print e
```

(iv) Control Transfer Analysis

- B1 -> B3 : taken when the condition 'a < d' is TRUE (goto 5).
- B1 -> B2 : fall-through when the condition 'a < d' is FALSE.
- B2 -> B4 : unconditional jump (goto 6).
- B3 -> B4 : fall-through (B3 ends right before B4).

(v) Final Block Structure

Four basic blocks B1–B4 are formed, each with a single entry (its leader) and a single exit (its last statement), satisfying the basic-block property required before flow-graph construct

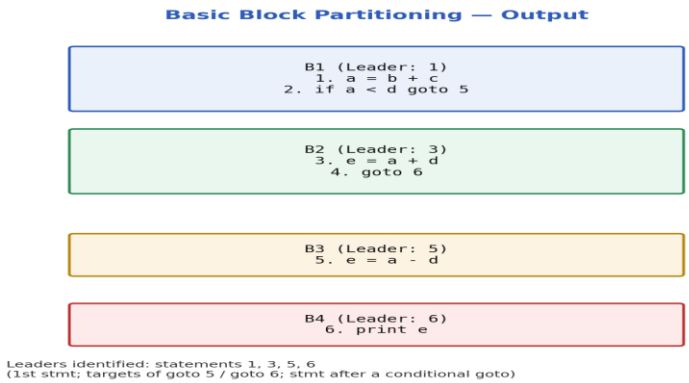
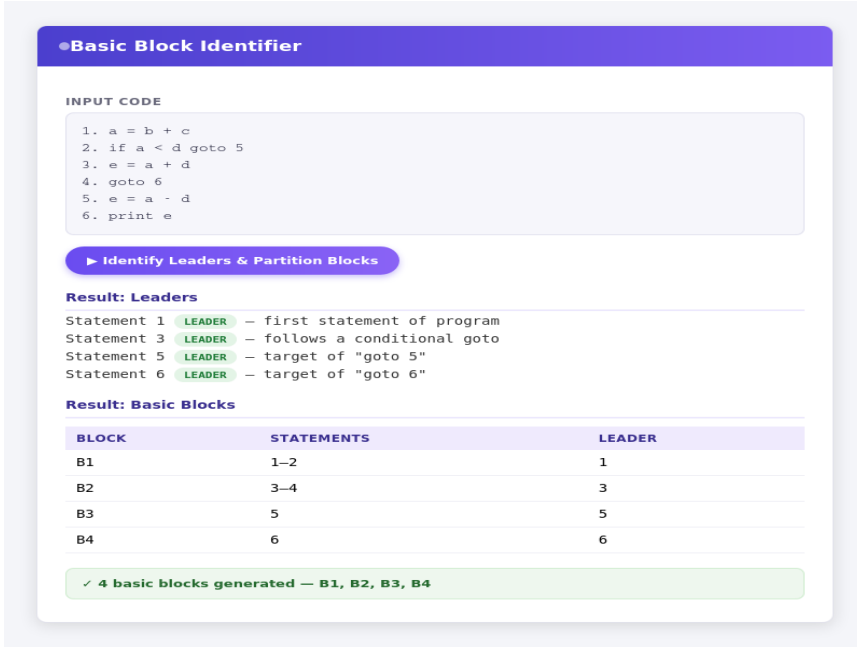


Fig. Q36 — Partitioned basic blocks B1–B4

BASIC BLOCK IDENTIFIER OUTPUT



Q37. Flow Graph Construction

Introduction

A flow graph represents the flow of control between the basic blocks of a program. Each node of the graph is a basic block, and a directed edge is drawn from block X to block Y if control can flow from the last statement of X directly to the first statement of Y.

(i) Node Creation for Each Block

Using the blocks identified in Q36, four nodes are created — one per basic block:

- Node B1: a = b + c ; if a < d goto 5
- Node B2: e = a + d ; goto 6
- Node B3: e = a - d
- Node B4: print e

(ii) Edge Construction Based on Control Flow

- B1 -> B2 (fall-through when a < d is false)
- B1 -> B3 (branch taken when a < d is true)
- B2 -> B4 (goto 6)
- B3 -> B4 (fall-through)

(iii) Entry and Exit Node Identification

Entry node: B1 — the first basic block, where program execution begins.

Exit node: B4 — the block containing the final statement (print e); no outgoing edges leave it.

(iv) Loop / Back-Edge Detection

A back edge is an edge whose target dominates its source (i.e., an edge that points to a block already visited higher up in the flow, forming a cycle). Examining the edges B1->B2, B1->B3, B2->B4 and B3->B4, every edge points strictly forward toward B4. No edge targets B1, B2 or B3 again, so there are no back edges and consequently no loop in this flow graph.

(v) Final Control Flow Graph

The final graph is a simple diamond-shaped DAG: B1 branches into B2 and B3, which both re-converge at B4.

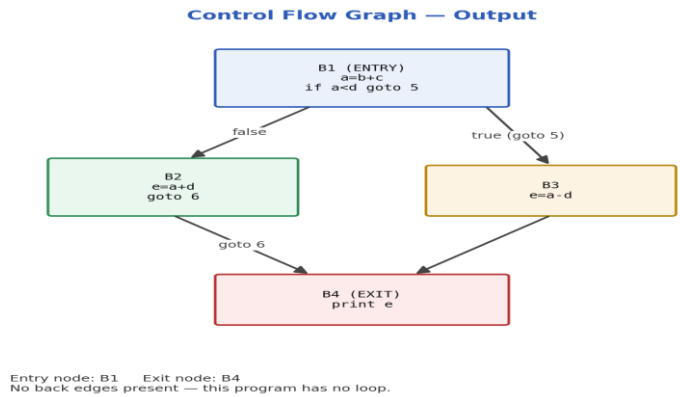


Fig. Q37 — Control flow graph for basic blocks B1–B4

FLOW GRAPH CONSTRUCTOR OUTPUT

Flow Graph Constructor

INPUT BASIC BLOCKS

```
B1: a=b+c ; if a<d goto 5
B2: e=a+d ; goto 6
B3: e=a-d
B4: print e
```

▶ Build Control Flow Graph

Result: Nodes

B1 (ENTRY) B2 B3 B4 (EXIT)

Result: Edges

FROM	TO	CONDITION
B1	B2	a<d is FALSE (fall-through)
B1	B3	a<d is TRUE (goto 5)
B2	B4	goto 6
B3	B4	fall-through

Back-edge / Loop Check

Scanning 4 edges... 0 back edges found.

✓ DAG constructed — Entry: B1, Exit: B4, Loops: none

Q38. Constant Propagation

Introduction

Constant propagation is a data-flow optimization that replaces a variable with its known constant value at compile time, wherever that value can be determined statically, eliminating unnecessary runtime computation.

```
a                                   =                                   5
b                   =               a                   +               3
c = b * 2
```

(i) Constant Identification

The statement 'a = 5' assigns a is assigned a compile-time constant, 5. Since a is not modified anywhere afterward, every later use of a can be replaced by 5.

(ii) Propagation Process

- Step 1: a = 5 -> a is a known constant (5).
- Step 2: b = a + 3 -> substitute a with 5 -> b = 5 + 3, which folds to b = 8 (now also a known constant).
- Step 3: c = b * 2 -> substitute b with 8 -> c = 8 * 2, which folds to c = 16.

(iii) Optimized Intermediate Code

```
a                                   =                                   5
b                                   =                                   8
c = 16
```

(iv) Updated Expression Values

- a = 5 (unchanged, source constant)
- b = 8 (was a + 3, i.e. 5 + 3)
- c = 16 (was b * 2, i.e. 8 * 2)

(v) Performance Improvement Explanation

All arithmetic (one addition and one multiplication) is evaluated once at compile time instead of at every program run, so the generated code simply loads the pre-computed constants. This reduces instruction count, removes runtime ALU operations, shrinks register pressure, and also creates more opportunities for further optimizations such as dead-code elimination on any now-unused temporaries.

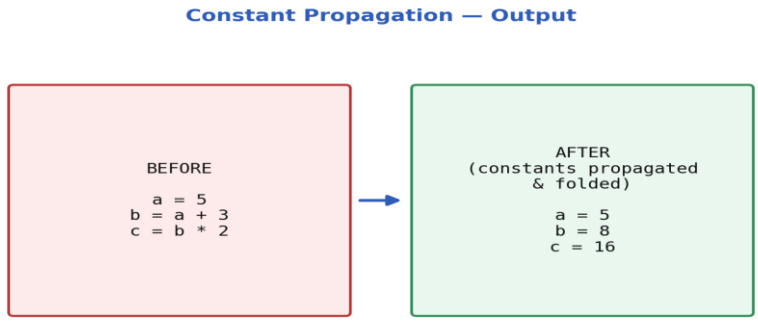


Fig. Q38 — Before / after constant propagation and folding

CONSTANT PROPAGATION OPTIMIZER OUTPUT

Constant Propagation Optimizer

INPUT CODE

```
a = 5
b = a + 3
c = b * 2
```

▶ Propagate Constants

Result: Propagation Trace

STEP	STATEMENT	SUBSTITUTION	FOLDED VALUE
1	a = 5	–	a = 5
2	b = a + 3	a → 5	b = 8
3	c = b * 2	b → 8	c = 16

Result: Optimized Code

BEFORE

~~a = 5~~
~~b = a + 3~~
~~c = b * 2~~

AFTER

```
a = 5
b = 8
c = 16
```

✓ 2 expressions folded — 0 runtime arithmetic operations remain

Q39. Dead Code Elimination

Introduction

Dead code elimination removes statements whose results are never used, i.e., variables that are assigned a value but are overwritten or the program ends before that value is read. Detecting dead code requires live-variable analysis.

```
a                                       =                                       5
b                                       =                                       10
a                                       =                                       7
print(b)
```

(i) Live Variable Analysis

Working backward from the last statement:

- print(b): b is used here, so b is LIVE just before this statement.
- a = 7: assigns a, but a is not used anywhere after this point (only b is printed) -> a is DEAD after this statement, so this assignment is dead.
- b = 10: b is used later by print(b) -> b is LIVE, so this assignment is NOT dead.
- a = 5: the value of a is immediately overwritten by 'a = 7' before ever being read -> this assignment is also DEAD.

(ii) Dead Statement Identification

Dead statements: 'a = 5' and 'a = 7' — both assign to a, and a is never read at any point in the program.

(iii) Code Elimination Process

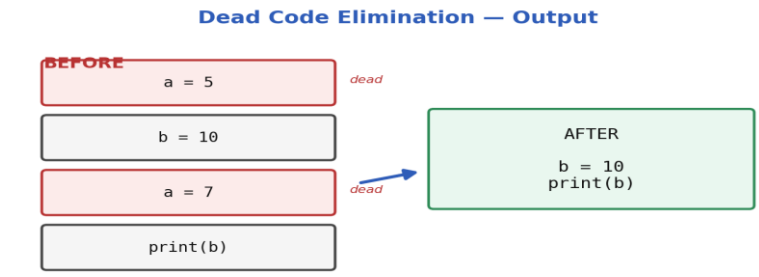
Both dead assignments to a are deleted since removing them does not change the observable behaviour of the program (the only observable effect is the value printed for b).

(iv) Optimized Final Code

```
b                                       =                                       10
print(b)
```

(v) Explanation of Optimization Benefit

Eliminating the two dead assignments to a removes unnecessary instructions and register/memory traffic, shrinking code size and reducing execution time, with zero impact on the program's output.



Live-variable analysis: 'a' is never used before being reassigned or after the program ends -> both writes to 'a' are dead.

Fig. Q39 — Dead statements identified and removed

DEAD CODE ELIMINATOR OUTPUT

Dead Code Eliminator

INPUT CODE

```
a = 5
b = 10
a = 7
print(b)
```

▶ Run Live Variable Analysis

Result: Live Variable Analysis (backward)

STATEMENT	LIVE-OUT	STATUS
a = 5	{ b }	DEAD
b = 10	{ b }	LIVE
a = 7	{ b }	DEAD
print(b)	{ }	LIVE

Result: Optimized Code

BEFORE

```
~~a = 5~~
b = 10
~~a = 7~~
print(b)
```

AFTER

```
b = 10
print(b)
```

✓ 2 dead statements removed ("a = 5", "a = 7")

Q40. Loop Optimization — Loop-Invariant Code Motion

Introduction

Loop-invariant code motion (LICM) hoists computations that produce the same result on every iteration of a loop out of the loop body, so they execute once instead of n times.

```
for (i = 0; i < n; i++) {
  x = a * b;
  y = x + i;
}
```

(i) Identification of Loop-Invariant Expressions

- x = a * b : invariant. Neither a nor b is assigned anywhere inside the loop body, so a*b evaluates to the same value on every iteration.
- y = x + i : NOT invariant, because i changes on every iteration (and, after hoisting, x is loop-invariant but i is not, so the sum still varies).

(ii) Movement Outside Loop

The invariant statement 'x = a * b' is moved to a preheader block immediately before the loop, executed exactly once before iteration begins.

(iii) Optimized Loop Structure

```
x = a * b;
for (i = 0; i < n; i++) {
  y = x + i;
}
```

(iv) Correctness Justification

Correctness holds because: (1) a and b are never redefined inside the loop, so x always holds the same value whether computed once before the loop or recomputed each iteration; (2) the statement has no side effects beyond assigning x; and (3) x is available (already computed) at every use inside the loop after hoisting. Hence the transformed program computes exactly the same values of x and y, and produces identical output to the original.

(v) Performance Improvement Analysis

Originally 'a * b' was recomputed n times (once per iteration), costing n multiplications. After hoisting, it is computed exactly once, costing 1 multiplication — an asymptotic reduction from O(n) to O(1) multiplications for that expression. For large n (e.g., large loop bounds), this removes a significant number of redundant arithmetic operations and reduces overall loop execution time.

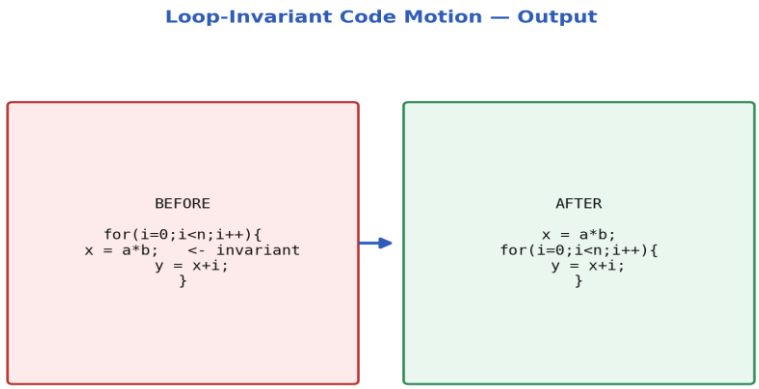


Fig. Q40 — Loop body before and after invariant code motion

LOOP-INVARIANT CODE MOTION OUTPUT

Loop-Invariant Code Motion

INPUT LOOP

```
for (i = 0; i < n; i++) {
  x = a * b;
  y = x + i;
}
```

▶ Detect & Hoist Invariants

Result: Invariance Check

STATEMENT	DEPENDS ON	STATUS
x = a * b	a, b (unchanged in loop)	INVARIANT
y = x + i	i (changes each iteration)	varies

Result: Optimized Loop

BEFORE

```
for (i=0;i<n;i++) {
  x = a * b;
  y = x + i;
}
```

AFTER

```
x = a * b;
for (i=0;i<n;i++) {
  y = x + i;
}
```

✓ 1 invariant expression hoisted — multiplications reduced from n to 1

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 — Exemplary	Level 3 — Proficient	Level 2 — Developing	Level 1 — Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understandin g with proper 1 tegration of most concepts	Basic understandin g with partial integration	Poor understandin g with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorith m Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validatio and justification	Good alysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis

Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation

RUBRICS FOR CALCULATION: